A

Project Report

on

Smart Expense System

B.Tech – IT , Sem-VI

Prepared By:

Arya Shah (IT-154)

Keval Hansaliya (IT-158)

Guided By:

Prof. R. A. Vyas

Dept. of Information Technology

Department of Information Technology

Faculty of Technology,

Dharmsinh Desai University

College Road, Nadiad – 387001

CANDIDATE'S DECLARATION

We Declare that the 6th-semester report entitled **"Smart Expense System"** is our work under the supervision of guide **Prof. R. A. Vyas.**

To the best of our knowledge, we further declare the report for B.Tech. VI semester does not contain part of the work which has been submitted either in this or any other university without proper citation.

Candidate's Signature
Candidate's Name: Arya Shah
Student ID: 24ITUOS912

Candidate's Signature
Candidate's Name: Keval Hansaliya
Student ID: 24ITUOS905

DHARMSINH DESAI UNIVERSITY

NADIAD-387001, GUJARAT

CERTIFICATE

This is to certify that the project out in the subject of Project-I, entitled "Pet Adoption System" and recorded in this report is a bonafide report of a work of

1) Arya Shah Roll No.: IT154 Student's ID: 24ITUOS912 2) Keval Hansaliya Roll No.: IT158 Student's ID: 24ITUOS905 of Department of Information Technology, Semester VI. They were involved in Project work during the academic year 2025-2026.

Prof. Viral H. Shah
(Project Guide)
Department of Information Technology,
Faculty of Technology,
Dharmsinh Desai University, Nadiad. Date:

Prof. (Dr.) Vipul K. Dabhi
Head , Department of Information Technology,
Faculty of Technology,
Dharmsinh Desai University, Nadiad. Date:

ACKNOWLEDGEMENT

It is indeed a great pleasure to express our thanks and gratitude to all those who helped us during this project. This project has given us a great opportunity to think, implement and interact with various aspects of the Software Development Life Cycle. We would like to acknowledge all the people who have helped us at one stage or another by providing the much-needed support, encouragement, and groundwork to complete our project.

We express a deep sense of gratitude towards our project guide **Prof. Viral H. Shah** towards his innovative ideas and earnest effort to make our project a success. It is his sincerity that prompted us throughout the project to do hard work using industry-adopted technologies. Our commitment to the application is the sole result of patience, hard work, and dedication being inspired by him.

A blend gratitude, pleasure, and great satisfaction are what we feel to convey our indebtedness to all those who have directly or indirectly contributed towards the completion of the project.

With Sincere Regards,
Arya Shah, Keval Hansaliya

Table of Contents

1. INTRODUCTION

1.1 PROJECT DETAILS

- The Expense Tracking System is a comprehensive web application designed to assist individuals and groups in managing their financial activities efficiently.
- It serves as a digital ledger that records income and expenses, categorizes transactions, and provides visual analytics to help users understand their spending habits.
- The system includes advanced features for budget management, alert generation, and shared group expense tracking, allowing multiple users to split bills and settle debts seamlessly.

1.2 PURPOSE

- The primary purpose of the Expense Tracking System is to provide a centralized platform for financial oversight, replacing manual methods like spreadsheets or paper logs.
- It aims to cultivate financial discipline by enabling users to set budget limits and receive real-time alerts when they are close to overspending.
- Additionally, the application resolves the complexity of shared finances (e.g., roommates or trips) by automatically calculating splits and tracking who owes whom.

1.3 SCOPE

- The scope of the project encompasses the development of a full-stack web application. Key modules include User Authentication (Registration, Login, Password Recovery), Category Management, and Transaction Recording (Income/Expense).
- It extends to analytical features such as Dashboard visualization, Filtering by date/category, and generating Monthly Reports.
- A significant portion of the scope is dedicated to Shared Group Management, which involves inviting members, recording group expenses, and automating equal expense splitting among members.

1.4 OBJECTIVE

- The objective is to develop a robust, user-friendly application that simplifies personal finance management and reduces the cognitive load of tracking daily expenditures.
- The system aims to ensure data accuracy in financial reporting and provide actionable insights through graphical representations (charts/graphs) of spending patterns.
- It seeks to facilitate transparency in group expenditures, ensuring fair and accurate division of costs among group members.

1.5 TECHNOLOGY AND LITERATURE REVIEW

1.5.1 Technology

- Frontend: The user interface is built using React.js, ensuring a responsive and dynamic single-page application (SPA) experience.
- Backend: The server-side logic is implemented using Node.js and Express.js, providing a lightweight, event-driven architecture for handling API requests and business logic efficiently.
- Database: A NoSQL database, MongoDB, is used to store user data, transaction records, and group information, allowing for flexible schema design and rapid scalability.

1.5.2 Literature Review

- The development of the Expense Tracking System is informed by existing financial management tools and best practices in full-stack web development.
- The decision to separate "Personal" and "Group" expenses was guided by user experience research suggesting that mixing these contexts often leads to confusion in standard banking apps.
- The implementation of "Budget Alerts" draws upon behavioral finance principles, where timely notifications are proven to help users curb impulsive spending.

2. PROJECT MANAGEMENT

2.1 FEASIBILITY STUDY

2.1.1 Technical Feasibility

- Technology Stack: The MERN Stack (MongoDB, Express, React, Node) is an industry-standard, JavaScript-everywhere solution with extensive community support, making it highly feasible for this project.
- Data Handling: MongoDB's document-oriented model is well-suited for storing complex, nested financial data (e.g., transactions inside user objects or groups).
- Security: JSON Web Tokens (JWT) and bcrypt will be utilized to handle authentication and password hashing, ensuring that user financial data remains private and secure.

2.1.2 Time Schedule Feasibility

- Development Time: The project is estimated to be completed within 3.5 to 4 months.
- Phasing: The modular nature of the requirements (R1 through R8) allows for parallel development, ensuring that core features (Auth, CRUD) can be finished early while complex features (Shared Groups, Reports) are developed subsequently.

2.1.3 Operational Feasibility

- Accessibility: As a web-based application, it requires no special hardware installation and is accessible from any device with a browser.
- User Adoption: The interface is designed to be intuitive, requiring minimal training for users to start tracking their expenses.

2.1.4 Implementation Feasibility

- Skill Set: The development team possesses the necessary skills in JavaScript, including ES6+ syntax, React Hooks, and Node.js asynchronous programming to execute the project.

- Tools: Standard development tools like VS Code, Postman (for API testing), and MongoDB Compass (for database visualization) are readily available and free to use.

2.2 PROJECT PLANNING

2.2.1 Project Development Approach and Justification

- Agile Methodology: An iterative Agile approach will be used. This allows for continuous testing and refinement of features like "Budget Alerts" and "Group Splitting" logic.
- Justification: Financial applications require high accuracy. Agile allows us to test each module (e.g., the calculation logic) independently before moving to the next, reducing the risk of critical bugs in the final release.

2.2.2 Project Plan

- Phase 1: Requirement Analysis & Design. Defining the schema, API endpoints, and UI wireframes.
- Phase 2: Core Development. Implementing User Auth, Category Management, and basic Transaction CRUD operations.
- Phase 3: Advanced Features. Implementing Budgeting logic, Shared Groups, and Reporting modules.
- Phase 4: Testing & Deployment. Unit testing, integration testing, and bug fixing.

2.2.3 Roles and Responsibilities

- Frontend Developer: Responsible for designing the UI using React, integrating APIs using Axios/Fetch, and managing application state (Redux/Context API).
- Backend Developer: Responsible for MongoDB schema design (Mongoose), REST API creation with Express, business logic implementation, and security.

2.2.5 Group Dependencies

- Technical Dependencies: The frontend is dependent on the backend API structure. Any change in the JSON response format from the Node.js server requires immediate updates

to the React components.

2.3 PROJECT SCHEDULING

Milestone	Start Date	End Date
Requirement Gathering & Design	13/12/2025	18/12/2025
Authentication & User Profile	19/12/2025	31/12/2025
Transaction & Category Modules	01/01/2026	15/01/2026
Budgeting & Alert Logic	16/01/2026	28/01/2026
Shared Group Management	29/01/2026	08/02/2026
Reporting & Dashboard	09/02/2026	14/02/2026
Testing & Final Documentation	15/02/2026	18/02/2026

3. System Requirements Study

3.1 Study of Current System

The "Current System" refers to the manual or semi-automated methods users currently employ to manage their finances. Before adopting the Smart Expense System, individuals and groups typically rely on:

- Manual Records: Tracking daily expenses using physical notebooks, diaries, or scraps of paper.
- Spreadsheets: Utilizing tools like Microsoft Excel or Google Sheets, which require manual data entry, formula maintenance, and lack mobile-friendly interfaces.
- Generic Tools: Using basic calculator apps or standard note-taking applications that store numbers but lack financial context or categorization.

- Verbal Tracking: Managing group expenses (e.g., trips, roommates) through verbal agreements or chat messages, leading to a lack of a permanent record.

3.2 Problems and Weaknesses of Current System

The existing systems suffer from significant limitations that hinder effective financial management:

1. Human Error: Manual calculations, especially for complex group splits (e.g., unequal sharing), are highly prone to mistakes.
2. Lack of Insight: Users cannot instantly visualize their financial health; they must manually aggregate data to see if they are over budget.
3. Data Fragmentation: Financial information is scattered across receipts, different apps, and mental notes, making consolidated reporting impossible.
4. Group Settlement Complexity: "Who owes whom" is difficult to track in active groups, often leading to social friction and unpaid debts.
5. Security Risks: Physical records can be lost, and simple spreadsheets often lack secure access controls like authentication.

3.3 User Characteristics

The system is designed to accommodate a wide range of users with varying levels of technical expertise:

- Students & Roommates: Primary users who need to split recurring costs like rent, utilities, and groceries.
- Travel Groups: Friends or colleagues planning trips who need a centralized ledger for shared travel expenses.
- Families: Households that require a joint view of expenses and budget tracking.
- Individual Users: People seeking to track personal spending habits, income, and savings goals.

  Competency: Users are assumed to have basic computer literacy, such as browsing the

web, registering accounts, and understanding basic financial concepts (income/expense).

3.4 Hardware and Software Requirements

To successfully develop, deploy, and use the Smart Expense System, the following configurations are required:

A. Software Requirements (Server & Development)

- Operating System: Windows, macOS, or Linux.
- Runtime Environment: Node.js (v14+).
- Database: MongoDB (Local or Cloud Atlas).
- Backend Framework: Express.js.
- Frontend Library: React.js (Vite).
- Version Control: Git.

B. Hardware Requirements (Client Side)

- Device: Desktop, Laptop, Tablet, or Smartphone.
- Processor: Minimum Intel Core i3 / AMD Ryzen 3 or equivalent mobile processor.
- RAM: Minimum 4GB.
- Connectivity: Active Broadband or Mobile Internet connection.

3.5 Constraints

- Internet Dependency: The system is a web application and requires a continuous internet connection to fetch real-time data and sync group activities.
- Email Service: The system relies on an external SMTP service (Gmail/Nodemailer) for OTP verification; if this service is down, new users cannot register.
- Browser Compatibility: The application is optimized for modern browsers (Chrome, Edge, Firefox) and may not fully support legacy browsers like Internet Explorer.

3.6 Assumptions

- Users will provide a valid email address during registration to receive the OTP.
- Users act in good faith when entering shared expenses and do not maliciously falsify amounts.
- The database server (MongoDB) maintains 99.9% uptime to ensure data availability.
- Users will keep their authentication tokens (JWT) secure and not share accounts.

4. System Analysis

4.1 Requirements of New System (SRS)

The Smart Expense System is designed to solve the problems of the current system by automating calculations, enforcing security, and providing real-time data visualization.

4.1.1 Functional Requirements

FR-01: User Registration & Verification

- Description: Allows a new user to register an account and triggers an email verification process to ensure authenticity.
- Input: username, email, password.
- Output: JSON Response { message: "OTP sent to email", email: "..." }.

FR-02: User Login

- Description: Authenticates a verified user and issues a secure session token (JWT) for accessing protected routes.
- Input: email, password.
- Output: JSON Response { _id, username, email, token }.

FR-03: Add Personal Transaction

- Description: Records a new financial entry (income or expense) linked to a specific category.
- Input: amount, type (income/expense), categoryId, date, description.

- Output: JSON Object of the created Transaction with populated Category details.

FR-04: Set Budget Limit

- Description: Defines or updates a monthly spending cap for a specific category to help users stay on track.
- Input: categoryId, amount (Monthly Limit).
- Output: JSON Object of the updated Budget document.

FR-05: Create Group

- Description: Initializes a new collaborative group for sharing expenses and auto-generates a unique join code.
- Input: name, type (e.g., "Trip", "Home").
- Output: JSON Group Object { _id, name, joinCode, members: [...] }.

FR-06: Add Shared Expense (Split)

- Description: Records a group expense and automatically calculates the share for each involved member based on the selected split type.
- Input: groupId, description, amount, splitType (equal/percentage/exact), splits (Array of shares).
- Output: JSON Object of the created GroupExpense.

FR-07: Calculate Group Debts (Split Report)

- Description: Computes the net financial standing of all group members, determining exactly who owes whom.
- Input: groupId.
- Output: JSON Object { balances: [{ userId, amount }], settlements: [{ from, to, amount }] }.

FR-08: Settle Up

- Description: Records a payment transaction between two members to clear or reduce an outstanding debt.
- Input: groupId, toUserId (Receiver), amount.
- Output: JSON Object of the created Settlement document.

4.1.2 Non-functional Requirements

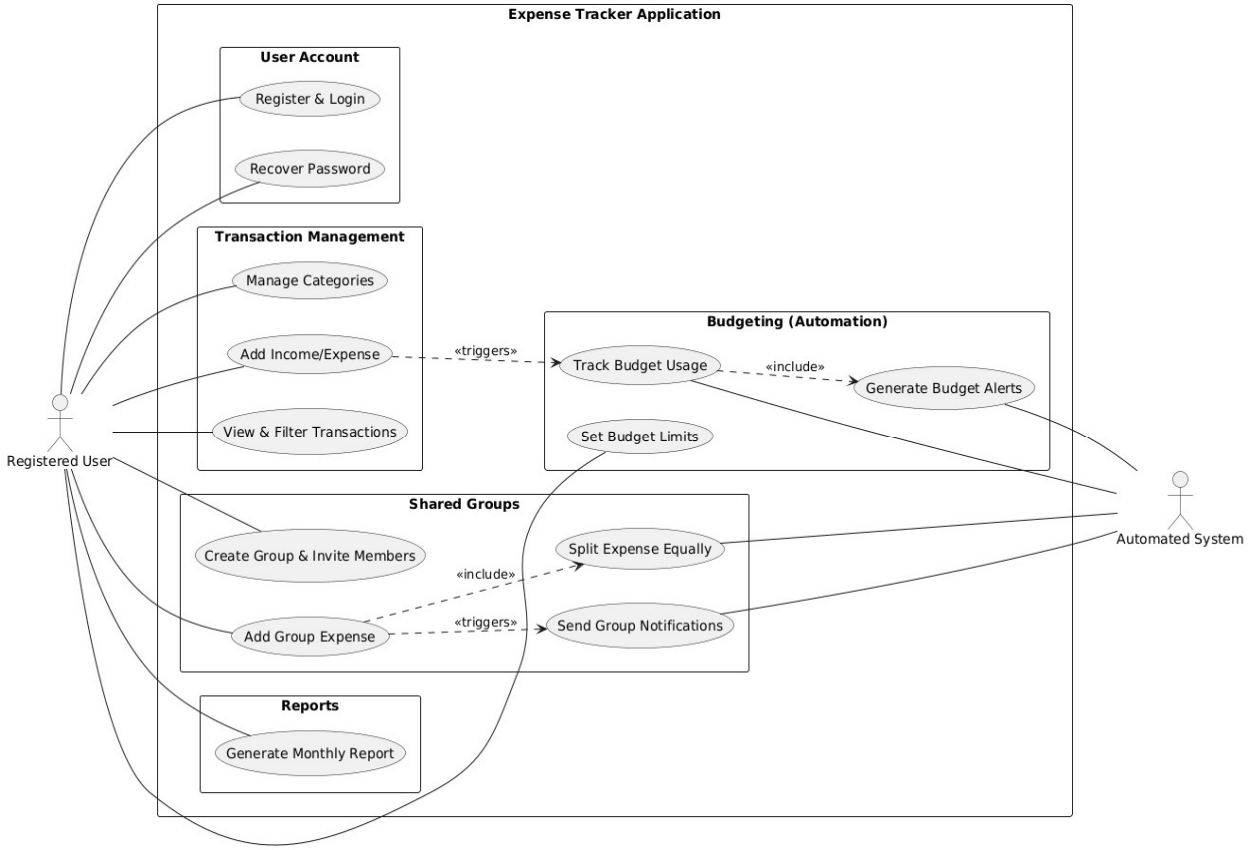
1. Security:
    - Data Protection: User passwords must be hashed using Bcrypt before storage in the database.
    - Access Control: API endpoints must be protected using JSON Web Tokens (JWT) to prevent unauthorized access.
    - Input Validation: The system must validate all inputs to prevent NoSQL injection and ensure data integrity.
2. Performance:
    - Response Time: Critical actions (like adding an expense) should be processed and acknowledged within 1 second.
    - Real-time Sync: Group notifications (via Socket.io) should appear on connected client devices within 500ms.
3. Reliability:
    - Data Integrity: The system must ensure that the sum of split shares in a group expense always equals the total amount.
    - Availability: The system should handle email delivery failures gracefully (e.g., allowing users to resend OTPs).
4. Scalability:
    - Concurrency: The Node.js backend should be capable of handling multiple concurrent requests from different users without blocking.

- Database: The MongoDB schema should support indexing on frequently queried fields (like userId and date) to maintain performance as data volume grows.

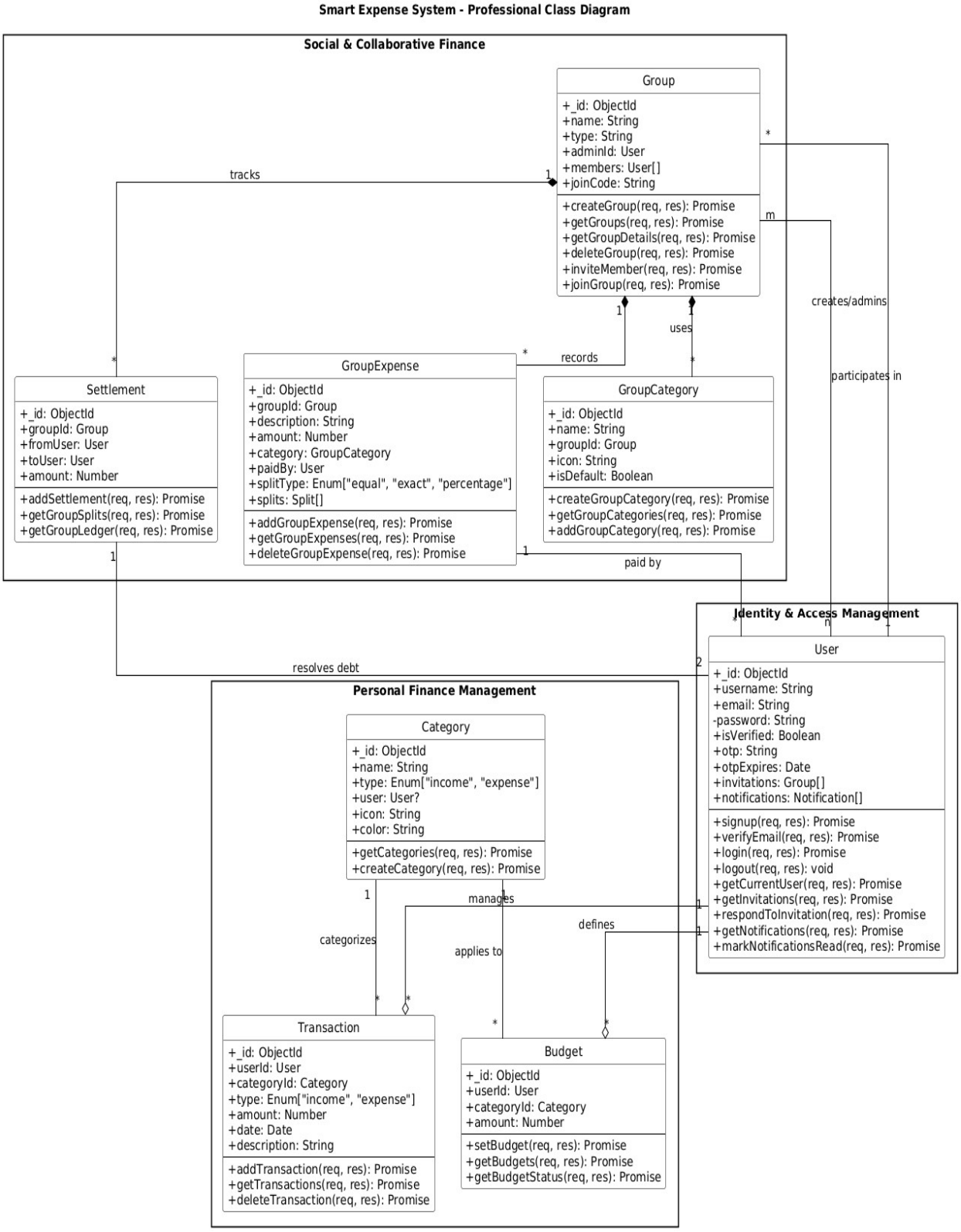
5. Usability:
   - Interface: The User Interface (UI) must be responsive, adapting seamlessly to desktop, tablet, and mobile screen sizes.
   - Feedback: The system must provide clear success/error messages (e.g., "Invalid Password", "Expense Added") to guide the user.

5. SYSTEM DESIGN

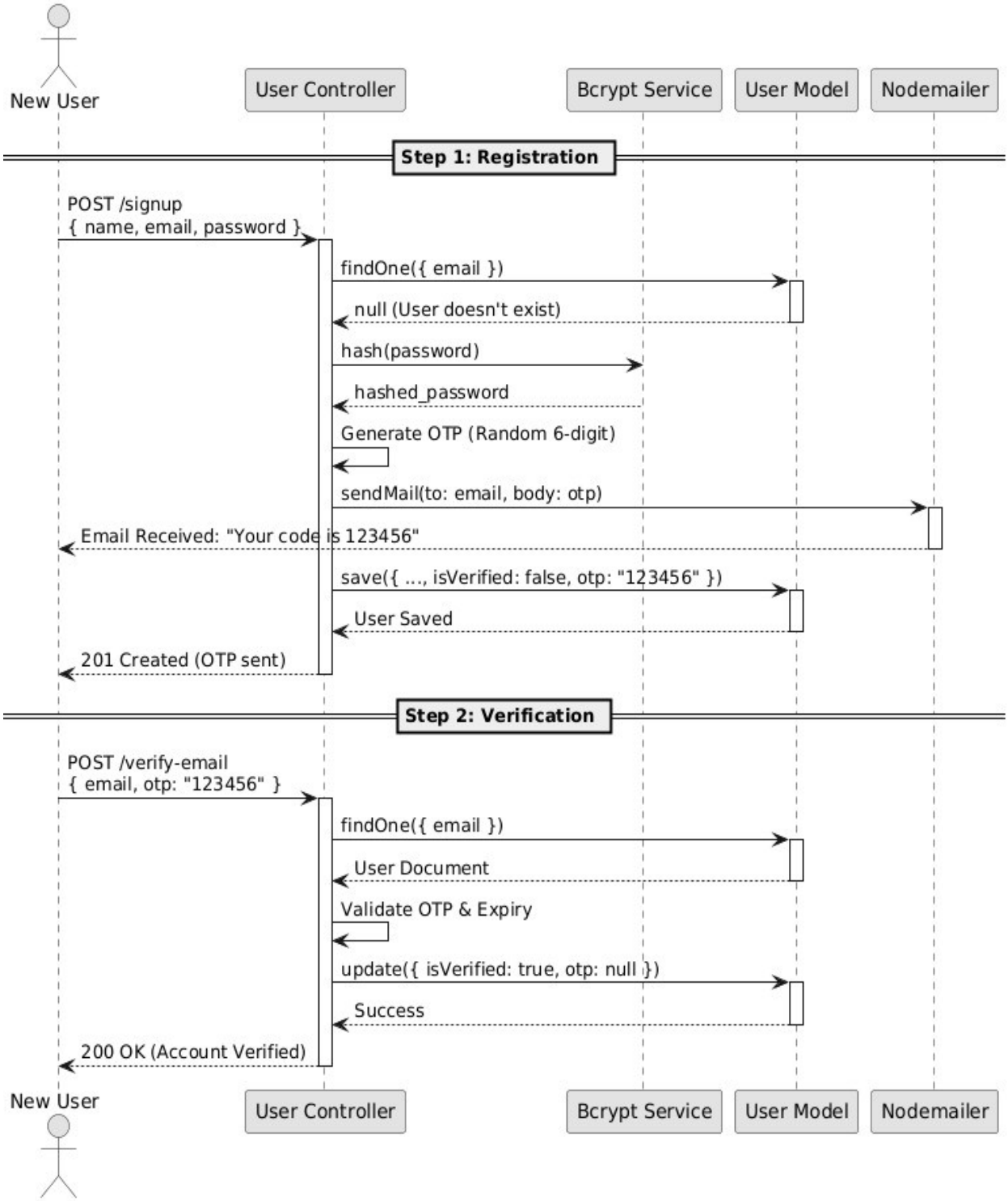
5.1 USE CASE DIAGRAM



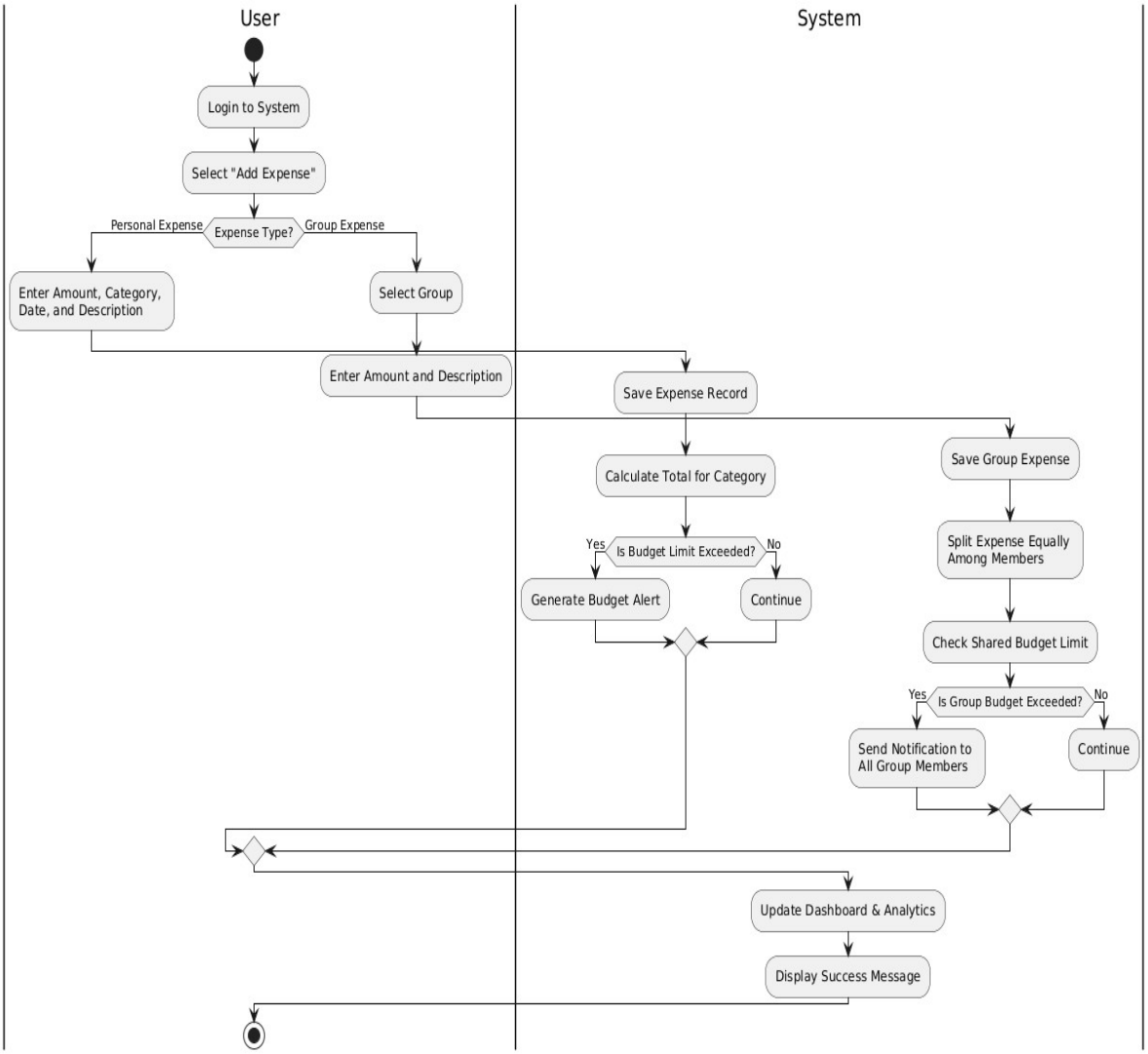
5.2 CLASS DIAGRAM



5.3 SEQUENCE DIAGRAM



5.4 ACTIVITY DIAGRAM



6. IMPLEMENTATION PLANNING

6.1 IMPLEMENTATION ENVIRONMENT

- Development Tools: Visual Studio Code (VS Code) is used as the primary Integrated Development Environment (IDE) for both frontend and backend development.
- Runtime Environment: Node.js is utilized as the runtime environment to execute the backend JavaScript code.
- Database Management: MongoDB Compass is used as the GUI tool to visualize and manage the NoSQL database collections.
- Version Control: Git is used for version control to track changes, with GitHub serving as the remote repository for code hosting and collaboration.
- API Testing: Postman is employed to test backend API endpoints (GET, POST, PUT, DELETE) before integrating them with the frontend.

6.2 PROGRAM/MODULES SPECIFICATION

1. User Management Module

- Functionality: Handles user registration, secure login, password recovery, and profile updates.
- Key Components: Registration form with validation, JWT (JSON Web Token) generation for session management, and profile editing interface.

2. Transaction Management Module

- Functionality: Allows users to record their financial activities, including income and expenses, and categorize them accordingly.
- Key Components: "Add Expense" form, "Add Income" form, transaction list view with edit/delete options, and category selection dropdowns.

3. Budget & Alert Module

- Functionality: Enables users to set monthly spending limits for specific categories and triggers alerts when limits are approached or exceeded.
- Key Components: Budget setting interface, background logic to calculate current spending vs. limit, and notification UI components.

4. Shared Group Module

- Functionality: Facilitates shared financial management by allowing users to create groups, invite members, and split expenses equally.
- Key Components: Group creation modal, member invitation via email, "Add Group Expense" logic, and a debt settlement view showing who owes whom.

5. Analytics & Reporting Module

- Functionality: Visualizes financial data to provide insights into spending habits and generates downloadable reports.
- Key Components: Dashboard with pie charts and bar graphs (using charting libraries), date range filters, and a "Download PDF/CSV" button for monthly reports.

7. Testing

7.1 Testing Plan

The testing plan for the Smart Expense System is designed to ensure that all functional requirements are met and that the system is robust, secure, and user-friendly. The plan outlines the scope, resources, and schedule for testing activities.

- **Objective:** To validate that the system accurately tracks expenses, manages group splits correctly, and secures user data.
- **Scope:**
  - **Backend:** Testing all API endpoints (Auth, User, Transactions, Groups) for correct responses and error handling.
  - **Database:** Verifying data integrity in MongoDB, ensuring relationships (e.g., Users to Groups) are maintained.
  - **Frontend:** Testing the React interface for responsiveness, form validation, and correct state updates.
- **Tools Used:**
  - **Postman / Thunder Client:** For manual API endpoint testing.
  - **Custom Scripts:** Using verify_backend.js to automate connectivity checks and route verification.
  - **Browser DevTools:** For debugging frontend console errors and network requests.

7.2 Testing Strategy

The strategy involves a bottom-up approach, starting with individual units and moving towards the complete system.

- **Unit Testing:**
  - Testing individual controller functions (e.g., verifying that the split calculation logic in addGroupExpense correctly divides amounts).
  - Verifying utility functions like createToken ensure they generate valid JWTs.
- **Integration Testing:**
  - Testing the interaction between the Frontend and Backend. For example, ensuring that clicking "Login" in the React app successfully sends a request to /api/users/login and stores the received token.
  - Verifying database interactions, such as checking if deleting a Group also deletes associated GroupExpenses.
- **Security Testing:**
  - Verifying that protected routes (e.g., Dashboard) cannot be accessed without a valid JWT token.
  - Ensuring passwords are not stored in plain text by checking the database entries.

7.3 Testing Methods

The project utilized a combination of manual and semi-automated testing methods:

- **Black Box Testing:** Focuses on the functionality without looking at the internal code structure.
  - *Example:* Trying to register with an invalid email format to see if the system rejects it, without checking the specific regex validator code.
- **White Box Testing:** Involves testing internal structures and workings of the application.
  - *Example:* Reviewing the verifyEmail controller to ensure the otp comparison logic accounts for expiration times.
- **Script-Based Verification:** Running the node verify_backend.js script to perform a health check on the server, ensuring all defined routes (Dashboard, Groups, Reports) are reachable and responding with status 200.

7.4 Test Cases

The following test cases were executed to validate the system's core functionality:

TC ID	Test Case Description	Input Data	Expected Result	Actual Result	Status
TC-01	User Registration	username: "Arya", email: "new@test.com", pass: "123456"	System sends OTP to email; Database saves user with isVerified: false.	OTP Received; User saved.	Pass
TC-02	Verify Email (Invalid OTP)	email: "new@test.com", otp: "000000" (Wrong OTP)	Error message: "Invalid or expired OTP".	Error: "Invalid or expired OTP".	Pass
TC-03	User Login	Valid Email & Password	Returns 200 OK; Response includes JWT Token.	Token received.	Pass
TC-04	Add Personal Expense	amount: 500, category:	Transaction saved;	Dashboard updated	Pass

		"Food", desc: "Pizza"	Balance updates on Dashboard.	instantly.	
TC-05	Create Group	name: "Goa Trip", type: "Travel"	Group created; Admin added to members; Unique joinCode generated.	Group ID & Code returned.	Pass
TC-06	Join Group (Via Code)	joinCode: "ABC-123" (Valid code)	User added to Group members list.	Success message received.	Pass
TC-07	Add Split Expense	amount: 1000, splitType: "Equal", members: 4	Expense saved; Each member assigned ₹250 share.	Math verified correct.	Pass
TC-08	Settle Up Debt	toUser: "User B", amount: 250	Settlement recorded; User A's debt to User B reduces by 250.	Balances updated in Report.	Pass
TC-09	Access Protected Route	Request /api/dashboard without Token	401 Unauthorized / "Not authorized, no token".	Access Denied.	Pass